

# Module 06 — Baseline Recommenders

models/baseline\_recommenders.py • 283 lines

## In one sentence

The four mandated baselines. These set the bar every other model must clear.

## 1. What this module does

Before building anything clever, you need to know what 'good' looks like. These four simple models provide that reference point.

The reason this matters: a deep learning model that cannot beat a plain top-sellers list is not worth its training cost or its serving latency. Building the baselines first means the comparison is honest.

As it turns out, one of these baselines - item-based collaborative filtering - beats the deep learning model on this dataset. That is a genuine finding, not a failure, and it is discussed openly in the evaluation.

## 2. Concepts covered

Every idea this module uses, in plain terms.

| Concept                  | In simple terms                               | Where it is used here                      |
|--------------------------|-----------------------------------------------|--------------------------------------------|
| Baseline model           | The simple thing you must beat                | All four models                            |
| Popularity ranking       | Show what sells best                          | build_popularity_model()                   |
| User-based CF            | Find similar people, show what they bought    | recommend_user_based()                     |
| Item-based CF            | Find similar products to what you liked       | recommend_item_based()                     |
| Cosine similarity        | Angle between two vectors                     | Both CF variants                           |
| Matrix factorisation     | Compress a big sparse grid into a few factors | create_svd_model()                         |
| Truncated SVD            | Keep only the strongest 50 patterns           | TruncatedSVD(n_components=50)              |
| Latent factors           | Hidden dimensions of taste                    | The 50 SVD components                      |
| Explained variance       | How much of the data the factors capture      | 33.09% here                                |
| Neighbourhood size       | How many similar users to listen to           | n_neighbours = 50                          |
| Computational complexity | How cost grows with size                      | $O(\text{users}^2)$ is the scaling problem |

## 3. How it works, step by step

- 1. Popularity model.** Count interactions per product, blend that with average rating and conversion rate, and sort. Each part is rescaled to 0-1 first so the weights mean what they say.
- 2. User-user similarity.** Compute cosine similarity between every pair of customers based on their rating vectors.
- 3. User-based recommendations.** Take the 50 most similar customers, and score products by what those customers rated highly, weighted by how similar they are.
- 4. Item-item similarity.** Compute cosine similarity between every pair of products.

**5. Item-based recommendations.** Score products by their similarity to what this customer already rated, weighted by those ratings.

**6. SVD.** Factorise the mean-centred rating matrix into 50 latent dimensions, then multiply the factors back out to fill in predictions for every empty cell.

## 4. Key functions

| Function                         | What it does                                     |
|----------------------------------|--------------------------------------------------|
| build_popularity_model()         | Ranks the catalogue by blended popularity        |
| recommend_popular()              | Top-N popular products the customer has not seen |
| create_user_similarity_matrix()  | Customer-to-customer similarity                  |
| recommend_user_based()           | Recommendations from similar customers           |
| create_item_similarity_matrix()  | Product-to-product similarity                    |
| recommend_item_based()           | Recommendations from similar products            |
| create_svd_model()               | Factorises the matrix into 50 latent dimensions  |
| create_predicted_rating_matrix() | Reconstructs the full dense score grid           |
| recommend_svd()                  | Top-N by predicted latent score                  |
| describe_items()                 | Maps IDs back to readable product rows           |

## 5. Inputs, outputs and how to run

|          |                                                                                  |
|----------|----------------------------------------------------------------------------------|
|          | Detail                                                                           |
| Inputs   | user_item_matrix, normalized_user_item_matrix                                    |
| Outputs  | popularity_model, user_similarity, item_similarity, svd_model, predicted_ratings |
| Results  | Item-CF NDCG@10 = 0.1044 • Popularity 0.0645 • SVD 0.0462                        |
| Runtime  | About 60 seconds                                                                 |
| Run with | python models/baseline_recommenders.py                                           |

## How the four baselines compare

| Model         | NDCG@10       | Coverage | Strength                       | Weakness                              |
|---------------|---------------|----------|--------------------------------|---------------------------------------|
| Item-based CF | <b>0.1044</b> | 22.2%    | best accuracy here             | cannot handle new products            |
| Popularity    | 0.0645        | 1.3%     | always works, instant          | identical for everyone                |
| SVD           | 0.0462        | 9.6%     | generalises across sparse data | harder to explain                     |
| User-based CF | not ranked    | n/a      | intuitive                      | $O(\text{users}^2)$ - breaks at scale |

## 6. Key code

### models/baseline\_recommenders.py — build\_popularity\_model()

```

34 | def build_popularity_model(interactions_df: pd.DataFrame) -> pd.DataFrame:
35 |     """Rank the catalogue by a blended popularity score."""
36 |     popularity_df = interactions_df.groupby("item_id").agg(
37 |         interaction_count=("item_id", "count"),
38 |         avg_rating=("rating", "mean"),
39 |         purchase_count=("purchase", "sum"),
40 |         total_revenue=("revenue", "sum"),
41 |     ).reset_index()
42 |
43 |     popularity_df["avg_rating"] = popularity_df["avg_rating"].fillna(0.0)
44 |     popularity_df["conversion_rate"] = (
45 |         popularity_df["purchase_count"] / popularity_df["interaction_count"]
46 |     )
47 |
48 |     def scale(series: pd.Series) -> pd.Series:
49 |         lo, hi = series.min(), series.max()
50 |         if hi > lo:
51 |             return (series - lo) / (hi - lo)
52 |         return pd.Series(0.0, index=series.index)
53 |
54 |     # Scale each component so the weights mean what they say; unscaled,
55 |     # interaction count would swamp average rating regardless of weight.
56 |     popularity_df["popularity_score"] = (
57 |         0.50 * scale(popularity_df["interaction_count"])
58 |         + 0.30 * scale(popularity_df["avg_rating"])
59 |         + 0.20 * scale(popularity_df["conversion_rate"])
60 |     )
61 |
62 |     popularity_df = popularity_df.sort_values(
63 |         "popularity_score", ascending=False
64 |     ).reset_index(drop=True)
65 |
66 |     save_pickle(popularity_df, "popularity_model.pkl")
67 |     return popularity_df

```

*Note that each component is rescaled to 0-1 before weighting.*

### models/baseline\_recommenders.py — recommend\_item\_based()

```

147 | def recommend_item_based(user_id: int, user_item_matrix: pd.DataFrame,
148 |                          item_similarity_df: pd.DataFrame, n_top: int = 10) -> list:
149 |     """Score candidates by similarity to the items this user rated."""
150 |     if user_id not in user_item_matrix.index:
151 |         raise ValueError("user_id " + str(user_id) + " not present in the rating matrix.")
152 |
153 |     user_ratings = user_item_matrix.loc[user_id]
154 |     rated_items = user_ratings[user_ratings > 0]
155 |
156 |     if rated_items.empty:
157 |         return []
158 |
159 |     scores = item_similarity_df[rated_items.index].to_numpy() @ rated_items.to_numpy()
160 |     scores = pd.Series(scores, index=item_similarity_df.index)
161 |
162 |     scores = scores.drop(rated_items.index, errors="ignore")
163 |     return scores.sort_values(ascending=False).head(n_top).index.tolist()

```

*The scoring is one matrix-vector product, not a Python loop.*

## 7. Things to remember

### The scaling problem you should be ready to discuss

The user-user similarity matrix is **O(users squared)**. At 5,766 customers it is 133 MB and fine. At 100,000 customers it would be 40 GB and simply will not fit in memory. The item-item matrix is **O(products squared)**, and catalogues grow far more slowly than customer bases. So **user-based CF is the first thing that breaks** as the platform grows.

**Why only 33% explained variance is acceptable**

The rating matrix is 99% empty. Most of the variance that is left after 50 components is noise in the cells nobody filled in. Adding more components would fit that noise rather than any real signal.

## 8. Likely questions

**Q. Why build baselines at all if you have a deep model?**

A. To know whether the deep model is worth anything. Without a baseline, an NDCG of 0.066 sounds fine - until you see that simply showing the best-sellers scores 0.064. The baselines are what make the comparison meaningful.

**Q. Why does the popularity model blend three things?**

A. Raw interaction count only measures how widely a product was *shown*. Adding average rating and conversion rate means a product has to be both seen and actually liked to reach the top.

**Q. Why does item-based CF beat user-based CF in practice?**

A. Products are more stable than people. A customer's taste and history change constantly; a product's set of buyers accumulates slowly. Item-item similarity is also far cheaper to maintain, because the catalogue grows more slowly than the user base.